

Solo E-Commerce Platform -- High Level Design (HLD)

Solo E-Commerce Platform

Document Version 1.0 | March 2026

Document Version: 1.1

Date: 09 May 2026

Author: Solo Engineering Team

Status: Final

1. Executive Summary

Solo is a direct-to-consumer e-commerce platform for the MENA market (primary geography: UAE) selling premium lifestyle, travel and home goods. The system is implemented as a headless three-tier web application: a **React + Vite** single-page storefront, a **NestJS REST API** backend, and a **PostgreSQL** relational database. It serves two primary user classes -- Customers (browse, checkout, account, loyalty) and Administrators (catalog, orders, returns, CMS, analytics, audit trail) -- through a single API surface.

The platform is hosted entirely on Microsoft Azure (region eastus2): the storefront on Azure Static Web Apps with a built-in CDN, the API on **Azure Container Apps**, the database on **Azure PostgreSQL Flexible Server**, and media on Azure Blob Storage. Continuous deployment is orchestrated by the Azure Developer CLI (azd) and GitHub Actions.

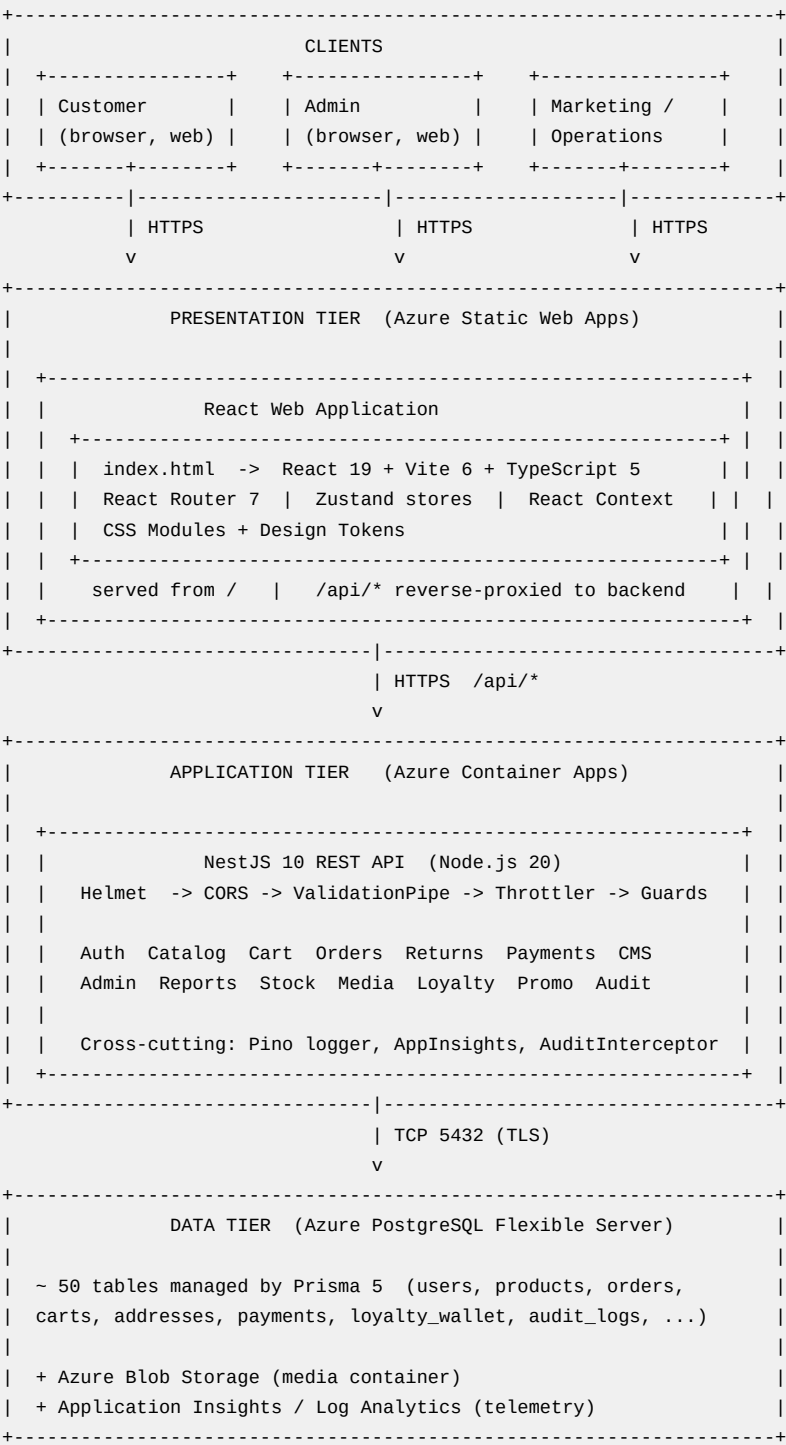
This HLD describes the macro-level decomposition, technology choices, deployment topology, key business flows, and the non-functional envelope within which the system must operate.

2. System Goals & Objectives

#	Goal	Success Measure
G1	Convert visitors to buyers	Conversion rate $\geq 1.5\%$ on product pages
G2	Maximise basket size	AOV trending up MoM
G3	Reduce checkout friction	Stripe + Tabby + Tamara + COD all live; ≤ 4 steps
G4	Earn repeat purchase	Loyalty wallet AED rebates; favorites; saved cards
G5	Stay VAT-compliant	Per-order VAT snapshot; downloadable PDF invoices
G6	Scale via B2B channel	Bulk-order RFQ flow with admin workflow
G7	Enable rapid merchandising	CMS-driven home, category landing, navigation
G8	Maintain auditability	Immutable audit trail of every admin write action

Primary objectives are: (a) ship a VAT-correct, bilingual-ready storefront with three live payment rails; (b) operate on a low-cost Azure footprint that scales horizontally; (c) keep the codebase strictly typed end-to-end (TypeScript on both tiers); (d) keep deployment a one-command operation (azd deploy).

3. High-Level System Architecture



Architecture style: Three-tier client-server with strict separation by network boundary. All inter-tier traffic is HTTPS/TLS; the SPA never speaks directly to the database. The backend is the sole steward of business invariants (stock, pricing, VAT, loyalty balances).

4. Key Components

4.1 Presentation Tier -- React Web Application

Aspect	Choice
Framework	React 19
Build tool	Vite 6
Language	TypeScript 5
Routing	react-router-dom v7
State	Zustand 5 + React Context (theme, toast)
HTTP	fetch-based client with auth interceptor in lib/apiClient.ts
Styling	CSS Modules + design tokens (styles/tokens.css)
Forms	React Hook Form
Charts (admin)	Recharts
Tests	Vitest 3 + React Testing Library + Playwright (e2e)
Token storage	localStorage (access + refresh)

Source location: frontend-react/

Folder layout (frontend-react/src):

```

components/ Reusable UI (Header, Footer, ProductCard, VariantSelector, ...)
config/      env.ts (VITE_API_URL, etc.)
contexts/    React contexts (theme, toast)
hooks/       useDebounce, useAuth, useCart, useFavorites, ...
lib/         apiClient + auth interceptor + currency/VAT formatters
pages/       Route components (one folder per page or grouped area)
stores/      Zustand stores (auth, cart, favorites, home)
styles/      CSS modules + design tokens
test/        Vitest setup + helpers
types/       Shared TS types

```

api/

The SPA reads `import.meta.env.VITE_API_URL` and prefixes all REST calls. It attaches Authorization: Bearer <accessToken> automatically; on a 401 it calls `/auth/refresh` once, retries the original request, and falls back to a login redirect if refresh fails.

4.2 Application Tier -- NestJS REST API

Aspect	Choice
Framework	NestJS 10
Runtime	Node.js 20 LTS
Language	TypeScript 5
ORM	Prisma 5
Auth	Passport-JWT (HS256), Argon2id password hashing
Validation	class-validator + global ValidationPipe (whitelist + transform)
Logging	nestjs-pino structured JSON logs
Telemetry	Application Insights SDK (boot-strapped in tracing.ts)
Rate limiting	@nestjs/throttler (global + auth-route override 5 / 15 min)
PDF	pdfkit (invoices)
Image processing	sharp (multi-size variants on upload)
Email	nodemailer over SMTP (Azure Communication Services compatible)

Module inventory (21+ modules, see 04 LLD for class-level detail):

AuthModule ProductsModule CategoriesModule BrandsModule

DesignersModule	CartModule	OrdersModule	ReturnsModule
PaymentsModule	StripeModule	TabbyModule	TamaraModule
CmsModule	ContentModule	MediaModule	FavoritesModule
PromoModule	LoyaltyModule	AdminModule	ReportsModule
StockModule	SettingsModule	AuditModule	CommonModule

Cross-cutting: Helmet, CORS allow-list (CORS_ORIGINS env), global
 HttpExceptionHandler, JwtAuthGuard, RolesGuard (RBAC: CUSTOMER,
 ADMIN, SUPER_ADMIN), AuditInterceptor (auto-logs every admin write to
 audit_logs).

4.3 Data Tier -- PostgreSQL Database

Aspect	Choice
Engine	PostgreSQL 14 (Azure Flexible Server)
Schema management	Prisma migrations (backend/prisma/migrations/)
Approx. tables	~50
Soft delete	deletedAt tombstones on products and similar
Snapshot fields	VAT, discount, shipping captured on each Order
Idempotency	stripe_events.id unique-key dedup
Backups	Azure-managed daily, point-in-time restore enabled

Migrations are applied automatically on container start
 (prisma migrate deploy && node dist/main.js) so that a fresh deploy is
 self-healing.

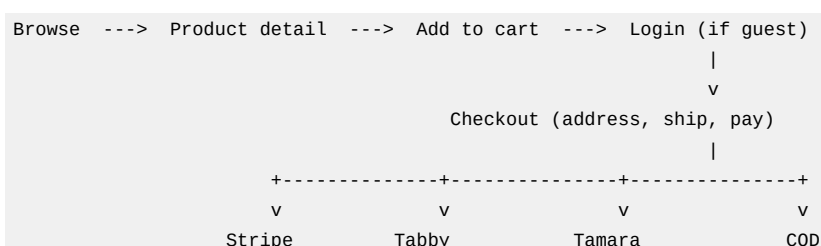
5. User Personas & Roles

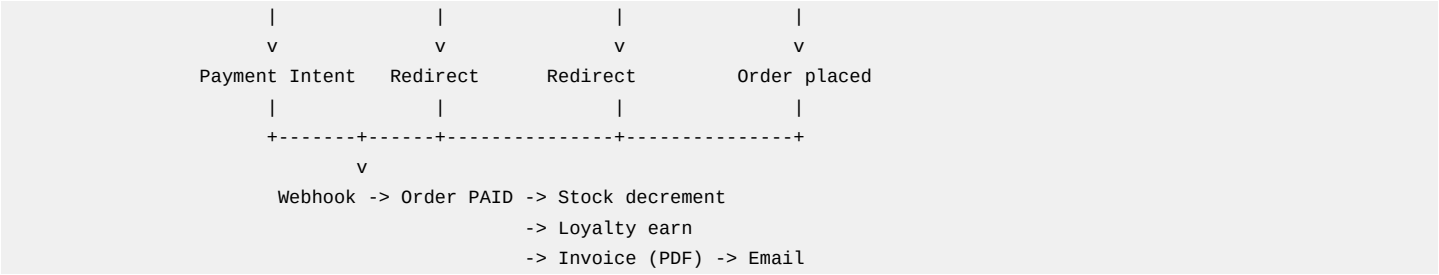
Persona	Typical Actions	Role
Sara (Shopper)	Browse, search, add to cart, checkout via Tabby	CUSTOMER
Khalid (Repeat)	Login, redeem loyalty AED, save cards, review favorites	CUSTOMER
Procurement	Submit bulk-order RFQ, await admin quote	CUSTOMER (RFQ)
Merchandiser	CMS -- banners, home sections, navigation, landing pages	ADMIN
Operations	Stock, orders, returns, customers, settings, reports	SUPER_ADMIN
Auditor	Read-only access to /admin/audit audit trail page	ADMIN

RBAC is enforced via the RolesGuard that reads @Roles(...) decorator
 metadata on controller methods. The UserRole enum is shared across DB,
 backend, and frontend type files.

6. Core Business Flows

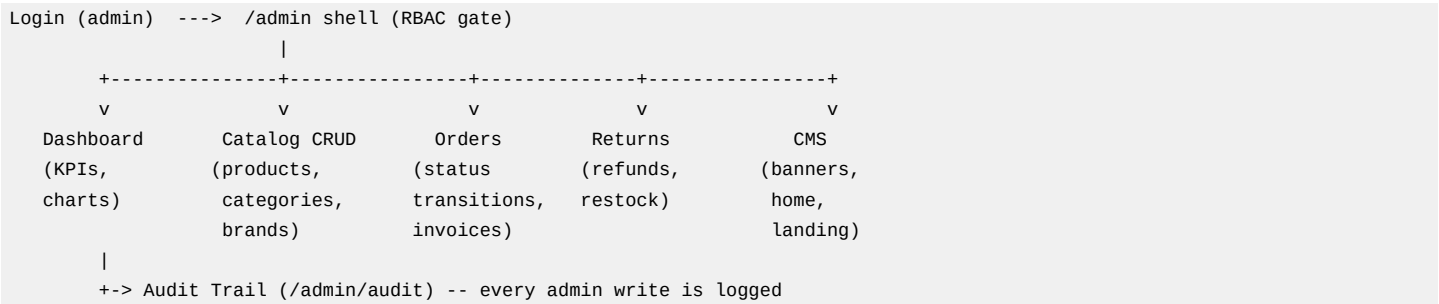
6.1 Customer Purchase Flow





Key invariant: stock decrement and order-state transitions happen inside a single Prisma transaction triggered by the verified webhook (Stripe) or by the success callback (Tabby/Tamara). Idempotency is enforced via the stripe_events table and provider-supplied reference IDs.

6.2 Admin Operations Flow



Every admin mutation passes through the global AuditInterceptor which records: actor (userId + email), action (PRODUCT_CREATED, ORDER_REFUNDED, ...), entity type & id, request metadata, and a JSON snapshot of the before/after state when relevant.

7. Integration Points

External System	Direction	Protocol	Purpose
Stripe	Out + Webhook in	HTTPS + signed webhook	Card payments
Tabby	Out + Webhook in	HTTPS + HMAC	BNPL (4 instalments)
Tamara	Out + Webhook in	HTTPS + HMAC	BNPL (3 / 6 instalments)
SMTP / ACS	Out	SMTPS	Transactional email
Azure Blob	Out	HTTPS	Media uploads & retrieval
Application Insights	Out	HTTPS	Telemetry / traces

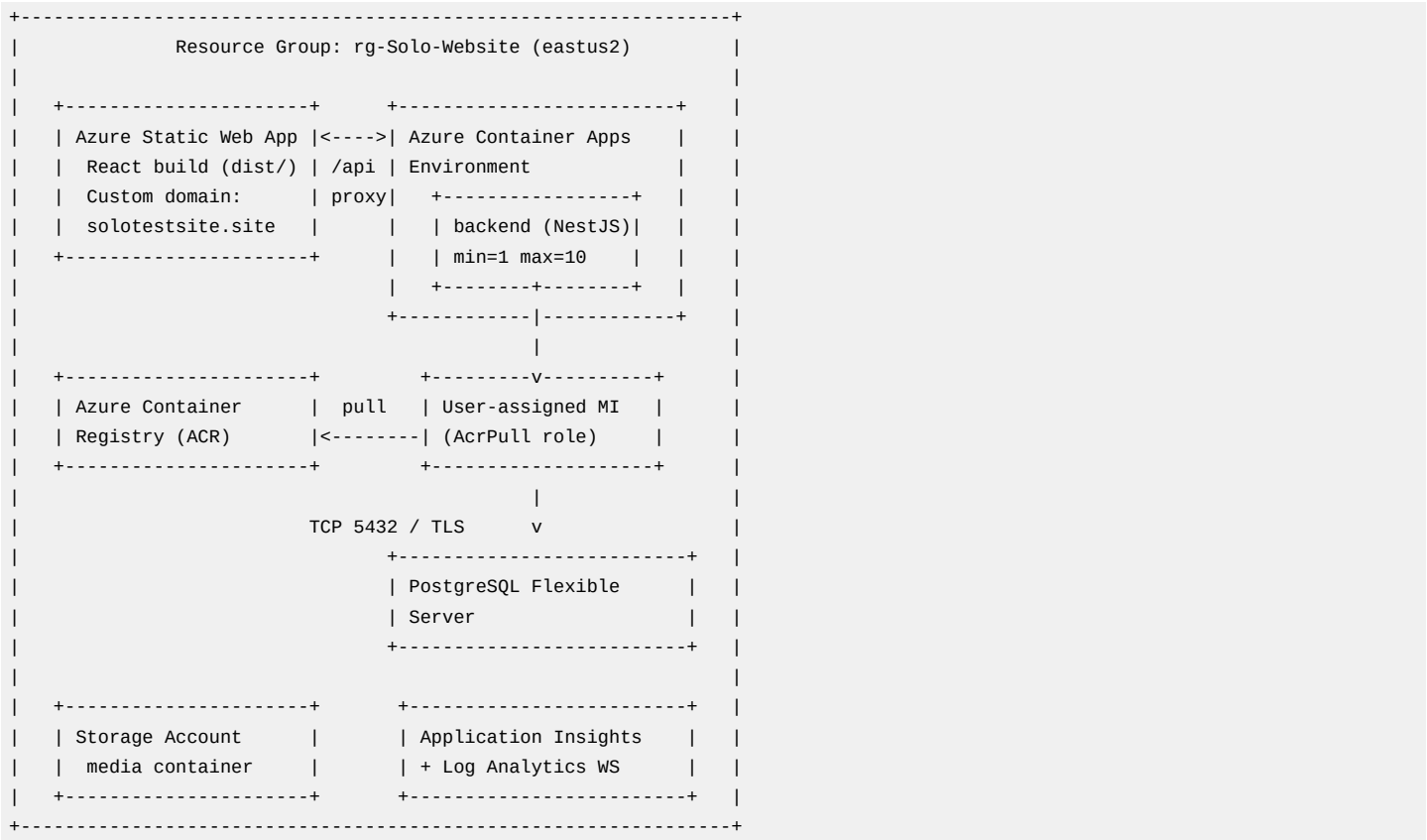
All third-party credentials live in Container App secrets (never in source). Webhooks are validated by signature (STRIPE_WEBHOOK_SECRET, provider HMAC keys); replays are rejected via the idempotency tables.

8. Non-Functional Requirements

Attribute	Target
Availability -- backend	99.5% monthly (single region)
Availability -- storefront	99.9% (CDN-served)
API latency p95 (read)	< 700 ms region-internal

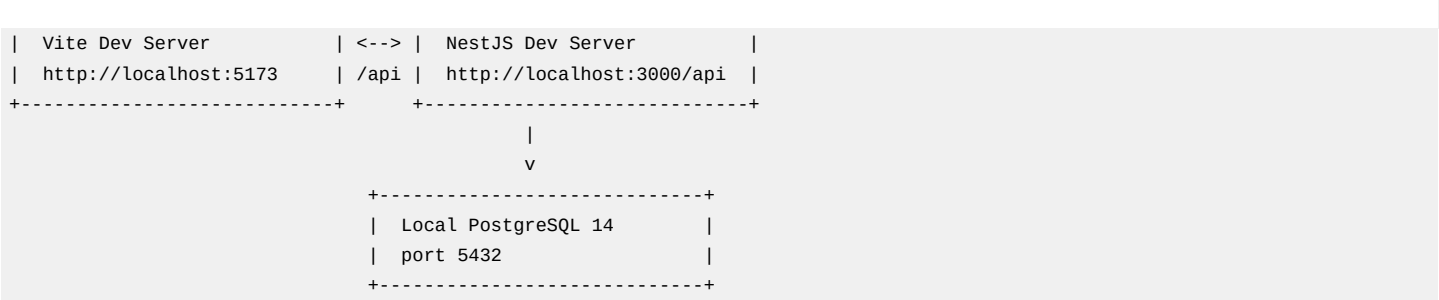
API latency p95 (write)	< 1.2 s
Cold-start (container)	< 5 s with min-replicas = 1
Throughput	100 RPS sustained per replica; horizontal scale to 10
Security	OWASP Top 10 covered (see 08 Security)
Observability	App Insights traces + Pino logs, correlated by trace ID
Recoverability	Daily automated PG backups; PITR; ACR holds image digests
Localisation	EN + AR data fields (UI strings English; AR planned)
Accessibility	Keyboard navigable, semantic HTML, alt text required on media
Compliance	UAE VAT 5% (configurable for KSA 15%), PCI-DSS via Stripe

9. Deployment Topology (Azure)



Provisioning: Bicep -- infra/main.bicep +
infra/main.parameters.json, orchestrated
via azure.yaml and azd up / azd deploy. See [09
Deployment & Operations](./09-deployment-and-operations.md) for the full
runbook.

Local development:



10. Technology Stack Summary

Layer	Technology	Version
Frontend Framework	React	19.x
Frontend Build	Vite	6.x
Frontend Language	TypeScript	5.x
Frontend Routing	react-router-dom	7.x
Frontend State	Zustand	5.x
Frontend HTTP	fetch + custom client	--
Frontend Forms	React Hook Form	7.x
Frontend Charts	Recharts	2.x
Backend Framework	NestJS	10.x
Backend Runtime	Node.js	20 LTS
Backend Language	TypeScript	5.x
ORM	Prisma	5.x
Database	PostgreSQL	14
Auth	Passport-JWT + Argon2id	--
Logging	nestjs-pino	--
Telemetry	Application Insights SDK	--
Container	Docker + ACR	--
Frontend Host	Azure Static Web Apps	--
Backend Host	Azure Container Apps	--
DB Host	Azure PostgreSQL Flexible Server	14
Media Storage	Azure Blob Storage	--
Payments	Stripe + Tabby + Tamara	--
Email	nodemailer + SMTP / ACS	--
IaC	Bicep	--
Deployment CLI	Azure Developer CLI (azd)	latest

11. Data Volume Estimates

Entity	Year-1 estimate	Year-3 estimate
Users	50 K	500 K
Products (incl. variants)	5 K	25 K
Orders	30 K	300 K
Order items	90 K	900 K
Loyalty transactions	60 K	600 K
Audit log rows (90-day retention)	~1 M	~10 M (with rolling prune)
Media blobs	30 K	200 K (~50 GB)

A single Azure PG Flexible Server SKU (B2ms) handles year-1 comfortably.
Year-3 likely needs D2s_v3 plus a read replica for catalog reads. Audit logs are pruned via a scheduled job after 90 days (with on-demand export to cold storage for compliance retention).

12. Risk & Mitigation

Risk	Impact	Likelihood	Mitigation
Single-region outage (eastus2)	High	Low	Documented DR runbook; planned active-passive multi-region
Stripe webhook lost / replayed	High	Low	Provider auto-retry + stripe_events idempotency

Out-of-stock race	Mediu	Medium	Stock decrement inside a Prisma transaction; 409 on conflict
JWT secret leak	High	Very low	Stored in ACA secrets; rotation runbook documented
Database CPU saturation	High	Medium	Per-route caching, read-replica plan, alert at 80% sustained
Third-party (BNPL) outage	Mediu	Medium	Toggle payment methods via site_settings to degrade gracefully
Admin privilege misuse	High	Low	RBAC + immutable audit_logs interceptor + retention
OWASP A01 (Broken Access Control)	High	Low	RolesGuard on every admin route; integration tests cover RBAC
OWASP A02 (Crypto Failures)	High	Low	Argon2id, HS256 JWT >= 32-byte secret, HTTPS-only
OWASP A03 (Injection)	High	Low	Prisma parameterised queries; class-validator whitelist
Cold-start latency spikes	Low	Medium	min-replicas = 1; warm-up endpoint hit by App Insights ping

End of High Level Design.